

Задание 3. Система симуляции фишинговой кампании

Оглавление:

[1. Ссылки](#)

[2. Описание проекта](#)

[3. Инструкция по запуску](#)

[4. Ход реализации](#)

1. Ссылки

GitHub-репозиторий: <https://github.com/eeyuymew/phish-simulator>

Видеодемонстрация: <https://disk.yandex.ru/i/6FUKR7t3sPfNbA>

2. Описание проекта

Учебный стенд «Phishing Simulator» разработан для проведения обучающих фишинговых рассылок внутри организации.

Стенд автоматизирует полный цикл: рассылку писем с персональными ссылками, фиксацию переходов по ним в базе данных и уведомление ответственного сотрудника (ИБ-отдела) в Telegram о каждом переходе.

Состав системы:

- Сервер Nginx - отдаёт страницу-уведомление, незаметно для пользователя передаёт сведения о переходе в трекер (механизм mirror);
- Tracker (Python/FastAPI) - фиксирует переходы в SQLite, отправляет уведомления в Telegram;
- Mailer (Python, CLI) - рассылка: чтение CSV, генерация уникальных токенов, отправка через SMTP;
- MailPit - встроенный SMTP-сервер с веб-интерфейсом для демонстрации (без внешней отправки);
- SQLite - хранение переходов и выданных токенов.

Весь стенд разворачивается одной командой `docker compose up -d`.

3. Инструкция по запуску

– Запуск проекта:

```
git clone https://github.com/eeuymew/phish-simulator.git
cd phish-simulator
docker compose up -d
```

Запускаются: nginx (порт 8084), tracker, mailpit (порт 8025).

Mailer - CLI-инструмент, запускается отдельно на шаге рассылки.

Список получателей лежит по пути - `mailer/data/users.csv`. Можно заменить своим в таком же формате

– Демонстрационная рассылка (через встроенный MailPit):

```
docker compose run --rm mailer
  --smtp-host mailpit --smtp-port 1025 \
  --from-addr security@test.ru \
  --base-url <IP машины со стендом>
```

Просмотр писем MailPit: `http://<IP>:8025`

– Рассылка на реальные адреса - указывается внешний SMTP-сервер:

внутренний relay (порт 25)

```
docker compose run --rm mailer --smtp-host <SMTP-сервер> --from-addr <отправитель> --base-url <IP стенда>
```

внешний сервер с шифрованием

```
docker compose run --rm mailer --smtp-host <SMTP-сервер> --smtp-port 587 --tls
```

```
docker compose run --rm mailer --smtp-host <SMTP-сервер> --smtp-port 465 --ssl
```

– Telegram-уведомления:

1. Создайте бота у @BotFather (/newbot), получите его токен.
2. Напишите боту любое сообщение и узнайте свой chat_id: откройте `https://api.telegram.org/bot<ТОКЕН>/getUpdates -> "chat":{"id":...}`.
3. В `docker-compose.yml` раскомментируйте строки в сервисе `tracker` и подставьте значения.
4. Обновите `docker compose up -d` - теперь после каждого перехода бот пришлёт сообщение.

– Проверка записи в БД:

```
docker compose exec tracker python -c "import sqlite3; [print(r) for r in sqlite3.connect('/app/data/tracker.db').execute('SELECT * FROM clicks')]"
```

4. Ход реализации

1. Запуск стенда одной командой.

Стенд разворачивается командой `docker compose up -d`.

```
(yoda@kali) - [~/tools/my-tools/trainings/phish-simulator]
$ docker compose up -d
[+] up 4/4
✓ Network phish-simulator_default Created
✓ Container tracker Started
✓ Container mailpit Started
✓ Container nginx Started

(yoda@kali) - [~/tools/my-tools/trainings/phish-simulator]
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
mailpit	axllent/mailpit	"/mailpit"	mailpit	4 seconds ago	Up 4 seconds (healthy)
nginx	nginx:alpine	"/docker-entrypoint...."	nginx	4 seconds ago	Up 3 seconds
tracker	phish-simulator-tracker	"uvicorn main:app --..."	tracker	4 seconds ago	Up 3 seconds

Поднимаются три сервиса: `nginx`, `tracker` и `MailPit`. Mailer вынесен в CLI-режим, так как рассылка - ручная операция.

2. Страница-уведомление и трекинг переходов.

Страница с текстом по заданию размещена в `nginx`.

Для фиксации переходов выбран механизм `mirror`:

письмо содержит «чистую» ссылку вида

`http://<IP>:8084/?email=...&token=...`, по которой пользователь переходит напрямую, а `nginx` копией запроса (`mirror /mirror_track`) в фоновом режиме передаёт её трекеру.

Скриншот конфигурации nginx:

```
imulador > nginx > ⚙ nginx.conf
http {
    upstream tracker_api {
        server tracker:8000;
        # Пул живых соединений upstream
        keepalive 16;
    }
    server {
        # Внутренний порт 80 наружу проброшен как 8084
        listen 80;

        # Поддержка кириллицы в HTML-странице
        charset utf-8;

        location / {
            # Прозрачный трекинг: ответ клиенту не ждёт tracker,
            # копия запроса уходит в фоновом режиме
            mirror /mirror_track;

            root /usr/share/nginx/html;
            index index.html;
        }

        location /mirror_track {
            # Прямой доступ к /mirror_track извне запрещён
            internal;

            # Без ретраев: иначе mirror-запрос дублируется
            proxy_next_upstream off;
            proxy_next_upstream_tries 1;

            proxy_http_version 1.1;
            proxy_set_header Connection "";

            # Все параметры (email, token) передаются в трекер как есть
            proxy_pass http://tracker_api/track$is_args$args;
            proxy_set_header X-Forwarded-For $remote_addr;
        }
    }
}
```

Ключевые параметры конфигурации:

- proxy_next_upstream off + proxy_next_upstream_tries 1 - запрет повторных попыток доставки mirror-запроса;
- X-Forwarded-For \$remote_addr - проброс реального IP клиента.

3. Tracker

Трекер реализован как чистый API-эндпоинт `/track`: принимает параметры ссылки, записывает событие, при необходимости шлёт уведомление. Порты трекера наружу не публикуются - добраться до него можно только через `nginx` внутри `compose`-сети.

Схема данных: таблица `clicks` (`email`, `token`, `ip`, `date` - для отображения). Включён режим `SQLite WAL` для защиты от блокировок при параллельных запросах.

Telegram-уведомление отправляется асинхронно (`httpx`) после успешной записи в БД и только если это не дубликат.

4. Mailer

Реализован как консольный инструмент с флагами запуска.

Возможности:

- **чтение CSV** - проверяется наличие колонки `email`; пустые строки пропускаются. Поля `first_name/last_name` опциональны - при отсутствии подставляется «Сотрудник».
- **Генерация токена** - `uuid.uuid4().hex[:8]`: 8 hex-символов (4,3 млрд комбинаций) - уникальность в рамках кампании обеспечивается колонкой `UNIQUE` в БД, а длина сохраняет ссылку читаемой;
- **Формирование ссылки** - `--base-url` нормализуется: пользователь указывает только IP/хост, схема `http://` и порт `:8084` добавляются автоматически (исключает опечатки и случайный порт). Итог:
<http://<IP>:8084/?email=<адрес>&token=<токен>>;
- **Персонализация шаблона** - в HTML-шаблоне (`mailer/data/letter.html`) заменяются плейсхолдеры `{{.FirstName}}`, `{{.LastName}}`, `{{.Email}}`, `{{.URL}}`. Шаблоны можно подкладывать свои - `mailer` выбирает указанный флагом `--template`, при единственном `.html` в каталоге - автоматически;
- **Формирование письма** - MIME-сообщение типа `multipart/alternative` (`email.mime`): HTML-часть кодируется в `utf-8`, заголовки `From/To/Subject` задаются явно. Отправитель - из флага `--from-addr` (по умолчанию вычисляется из домена SMTP-хоста: `security@<домен>`).

Учёт результатов. По каждому получателю (вне зависимости от успеха) выполняется INSERT в `mailer/data/tokens.db`: email, токен, итоговая ссылка, имя шаблона, SMTP-сервер, статус `sent_ok` (1/0), время выдачи.

Основные флаги запуска `mailer`:

- `--smtp-host` - адрес SMTP-сервера (обязательный); `--smtp-port` - порт (по умолчанию 25);
- `--from-addr` - отправитель (по умолчанию `security@<домен smtp-host>`);
- `--base-url` - IP/DNS хоста стенда для генерации ссылок;
- `--template` - HTML-шаблон из `mailer/data/` (если в каталоге один файл - выбирается автоматически);
- `--csv` - файл со списком получателей (по умолчанию `mailer/data/users.csv`);
- `--tls` / `--ssl` - режимы STARTTLS / SMTPS;
- `--dry-run` - показать ссылки без отправки писем.